*k*NN
(*k*-Nearest Neighbors)

Boris Velichkov

Machine Learning

Supervised Learning

Unsupervised Learning

Classification

Humidity (%)	Wind (km/h)	Rain (mm)	Class - Temp. (°C)
80	15	2.0	**Mild**
50	5	0.0	**Hot**
85	20	5.5	**Cold**
60	10	0.0	**Mild**
90	25	8.0	**Cold**

$x_t = (x_{t_{Humidity}} = 85, x_{t_{Wind}} = 25, x_{t_{Rain}} = 6.0)$

$\mathbf{Class}(x_t) = y, \quad y \in \{"Cold", "Mild", "Hot"\}$

Regression

Humidity (%)	Wind (km/h)	Rain (mm)	Class - Temp. (°C)
80	15	2.0	**22**
50	5	0.0	**30**
85	20	5.5	**18**
60	10	0.0	**25**
90	25	8.0	**10**

$x_t = (x_{t_{Humidity}} = 85, x_{t_{Wind}} = 25, x_{t_{Rain}} = 6.0)$

$\mathbf{Class}(x_t) = y, \quad y \in \mathbb{R}$

Clustering

Humidity (%)	Wind (km/h)	Rain (mm)
80	15	2.0
50	5	0.0
85	20	5.5
60	10	0.0
90	25	8.0

$\forall x_i \in \mathbb{X}, \quad x_i = (x_{i_{Humidity}}, x_{i_{Wind}}, x_{i_{Rain}})$

$\mathbb{X} = \{x_1, x_2, \dots, x_k\}$

$\mathbf{Clusters}(\mathbb{X}) = \{C_1, C_2, \dots, C_k\}$

Machine Learning

Supervised Learning

Classification

Humidity (%)	Wind (km/h)	Rain (mm)	Class - Temp. (°C)
80	15	2.0	**Mild**
50	5	0.0	**Hot**
85	20	5.5	**Cold**
60	10	0.0	**Mild**
90	25	8.0	**Cold**

$x_t = (x_{t_{Humidity}} = 85, x_{t_{Wind}} = 25, x_{t_{Rain}} = 6.0)$

$\mathbf{Class}(x_t) = y, \quad y \in \{"Cold", "Mild", "Hot"\}$

Regression

Humidity (%)	Wind (km/h)	Rain (mm)	Class - Temp. (°C)
80	15	2.0	**22**
50	5	0.0	**30**
85	20	5.5	**18**
60	10	0.0	**25**
90	25	8.0	**10**

$x_t = (x_{t_{Humidity}} = 85, x_{t_{Wind}} = 25, x_{t_{Rain}} = 6.0)$

$\mathbf{Class}(x_t) = y, \quad y \in \mathbb{R}$

Unsupervised Learning

Clustering

Humidity (%)	Wind (km/h)	Rain (mm)
80	15	2.0
50	5	0.0
85	20	5.5
60	10	0.0
90	25	8.0

$\forall x_i \in \mathbb{X}, \quad x_i = (x_{i_{Humidity}}, x_{i_{Wind}}, x_{i_{Rain}})$

$\mathbb{X} = \{x_1, x_2, \dots, x_k\}$

$\mathbf{Clusters}(\mathbb{X}) = \{C_1, C_2, \dots, C_k\}$

Global Learning vs Local Learning

- **Global Learning**: Learning from all instances in the dataset.
  - Linear Regression
  - Naïve Bayes Classifier
- **Local Learning**: Learning from some of the instances in the dataset.
  - *k*NN

** Local & Global Learning is different from Local & Global Search!*

Instance-Based Learning vs Model-Based Learning

- **Instance-Based Learning**: Use the entire dataset as the model.
  - *k*NN
- **Model-Based Learning**: Use the training data to create a model that has parameters learned from the training data.
  - Linear Regression
  - Naïve Bayes Classifier

Instance-Based Learning vs Model-Based Learning

Model-based learning techniques

Use the input data

$$\begin{bmatrix} x_{1,0} & x_{1,1} & \dots & x_{1,n} \\ x_{2,0} & x_{2,1} & \dots & x_{2,n} \\ \vdots & \vdots & \vdots & \vdots \\ x_{m,0} & x_{m,1} & \cdots & x_{m,n} \end{bmatrix} \text{ and } \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_m \end{bmatrix}$$

To learn a set of parameters

$$\begin{bmatrix} \theta_0 & \theta_1 & \cdots & \theta_n \end{bmatrix}$$

Which yield a **generalized** function

$$f(x) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_n x_n$$

Capable of predicting values or classes on new input data

$$f(x_i) = 39$$

$$f(x_j) = 1$$

Instance-based learning techniques

Store the input data

$$\begin{bmatrix} x_{1,0} & x_{1,1} & \dots & x_{1,n} \\ x_{2,0} & x_{2,1} & \dots & x_{2,n} \\ \vdots & \vdots & \vdots & \vdots \\ x_{m,0} & x_{m,1} & \cdots & x_{m,n} \end{bmatrix} \text{ and } \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_m \end{bmatrix}$$

When asked to predict a new value (a query)

$$y_i = ?$$

Search for similar data points previously stored

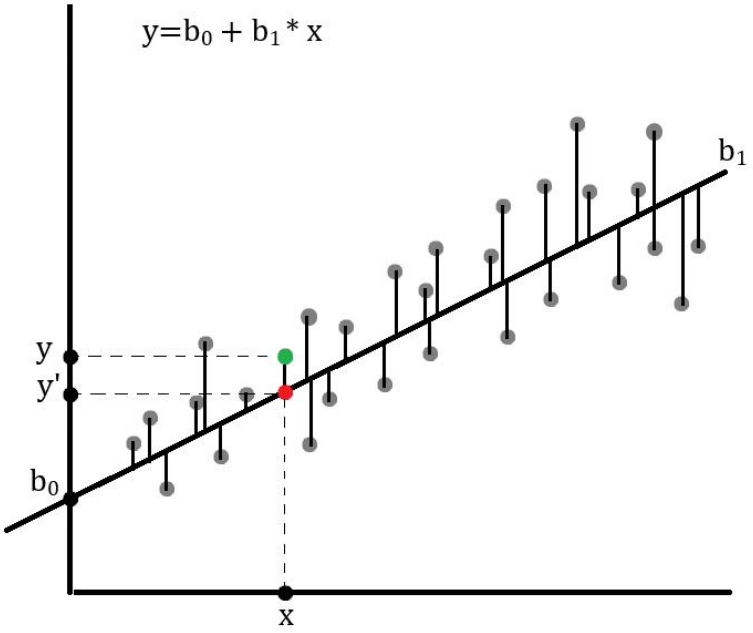
$$\begin{bmatrix} x_{4,1} & x_{4,2} & \cdots & x_{4,n} \\ x_{9,1} & x_{9,1} & \cdots & x_{9,n} \\ x_{15,1} & x_{15,1} & \cdots & x_{15,n} \end{bmatrix} \text{ and } \begin{bmatrix} y_4 \\ y_9 \\ y_{15} \end{bmatrix}$$

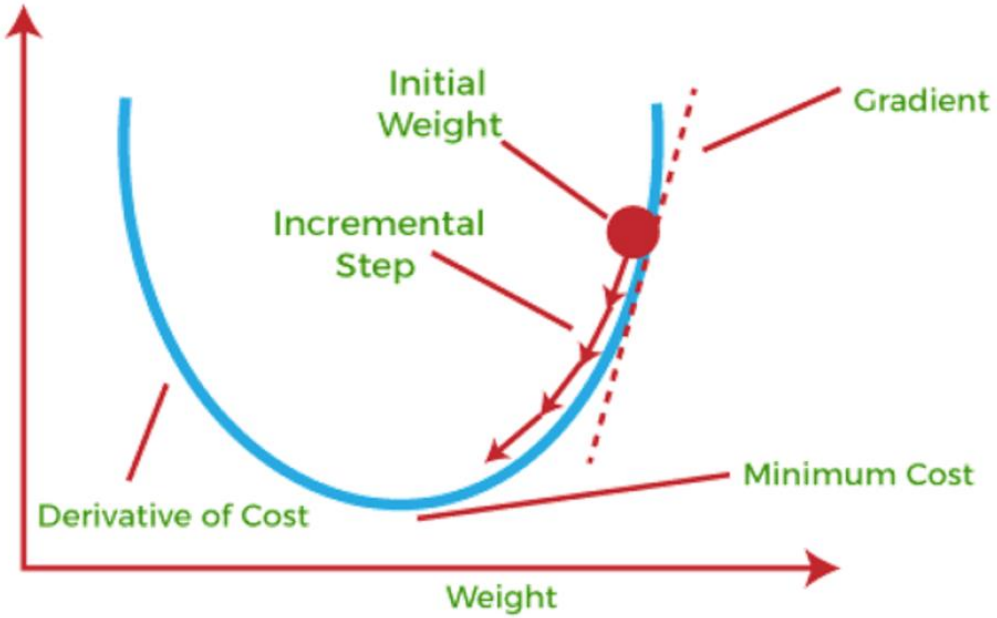
And use them to generate your prediction

$$y_i = \frac{y_4 + y_9 + y_{15}}{3}$$

Linear Regression and Gradient Descent

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - y_i')^2$$





x = x - learning_rate * gradient

Parameters vs Hyperparameters

Aspect	Parameters	Hyperparameters
Set by	Learned from training data.	Manually chosen or tuned.
Examples in Linear Regression	Weights w, Bias b.	Learning rate, optimization algorithm.
Examples in kNN	None (no learned parameters).	k, distance metric.

Lazy Learning vs Eager Learning

- Lazy vs. eager learning
  - Lazy learning (e.g., instance-based learning): Simply stores training data (or only minor processing) and waits until it is given a test tuple
  - Eager learning (eg. Decision trees, SVM, NN): Given a set of training set, constructs a classification model before receiving new (e.g., test) data to classify
- Lazy: less time in training but more time in predicting
- Accuracy
  - Lazy method effectively uses a richer hypothesis space since it uses many local linear functions to form its implicit global approximation to the target function
  - Eager: must commit to a single hypothesis that covers the entire instance space

Lazy Learner: Instance-Based Methods

- Instance-based learning:
  - Store training examples and delay the processing ("lazy evaluation") until a new instance must be classified
- Typical approaches
  - k-nearest neighbor approach
    - Instances represented as points in a Euclidean space.
  - Locally weighted regression
    - Constructs local approximation
  - Case-Based Reasoning (CBR)

kNN

- All instances correspond to points in the ***n-D*** space
- The nearest neighbor are defined in terms of Euclidean distance, ***dist(x_1, x_2)***
- Target function could be *discrete* or *real* value
- For *discrete-valued*, ***k*-NN** returns the *most common value* among the ***k*** training examples nearest to the ***test instance***.

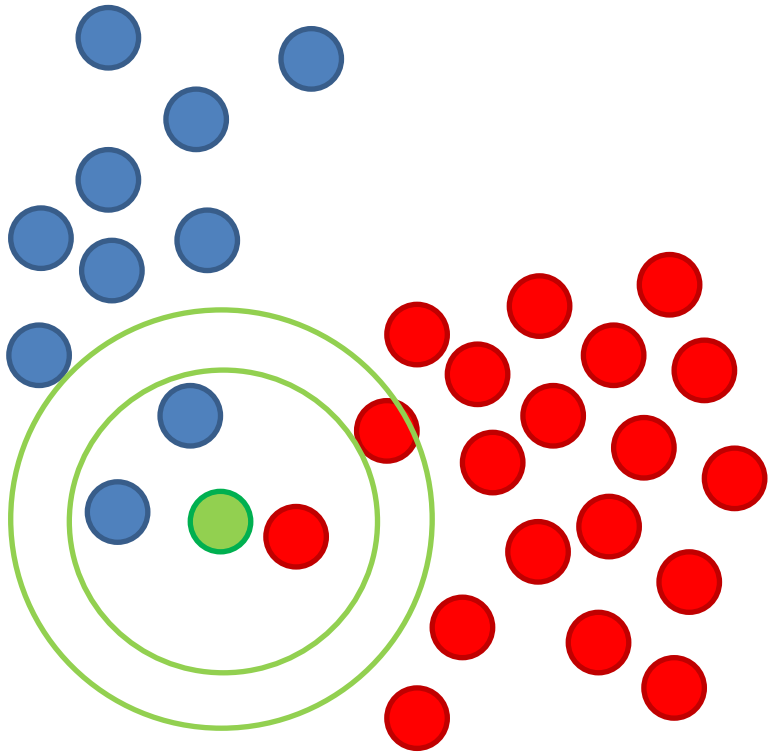
kNN

- ***K*-NN** for *real-valued* prediction for a given unknown tuple
  - Returns the *mean values* of the ***k*** *nearest neighbors*
- Distance-weighted nearest neighbor algorithm
  - Weight the contribution of each of the ***k*** neighbors according to their distance to the query $\boldsymbol{x_q}$
    - Give greater weight to closer neighbors
- Robust to noisy data by averaging *k*-nearest neighbors
- Curse of dimensionality: distance between neighbors could be dominated by irrelevant

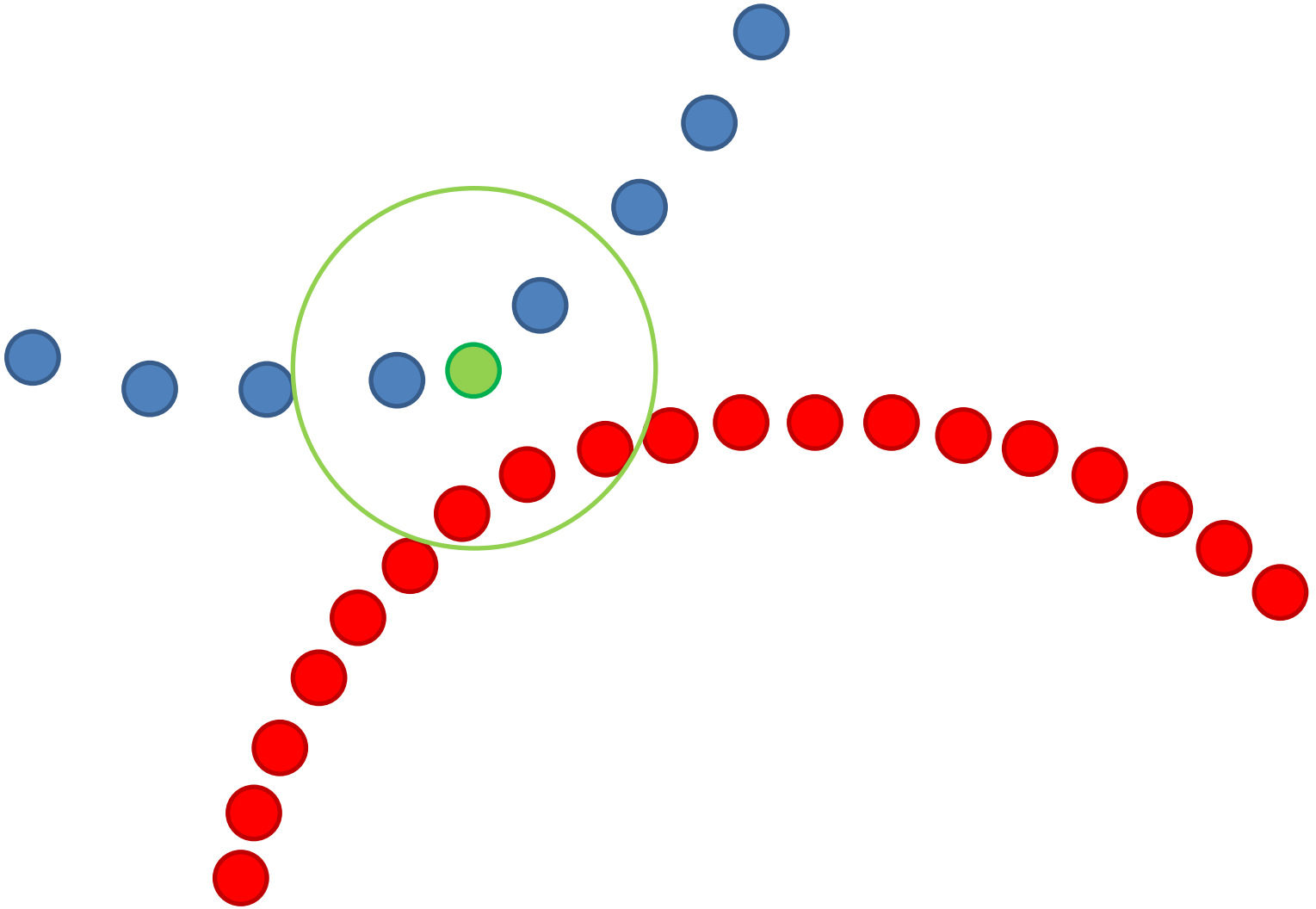
kNN

- Kernel estimation
  - k-nearest neighbor

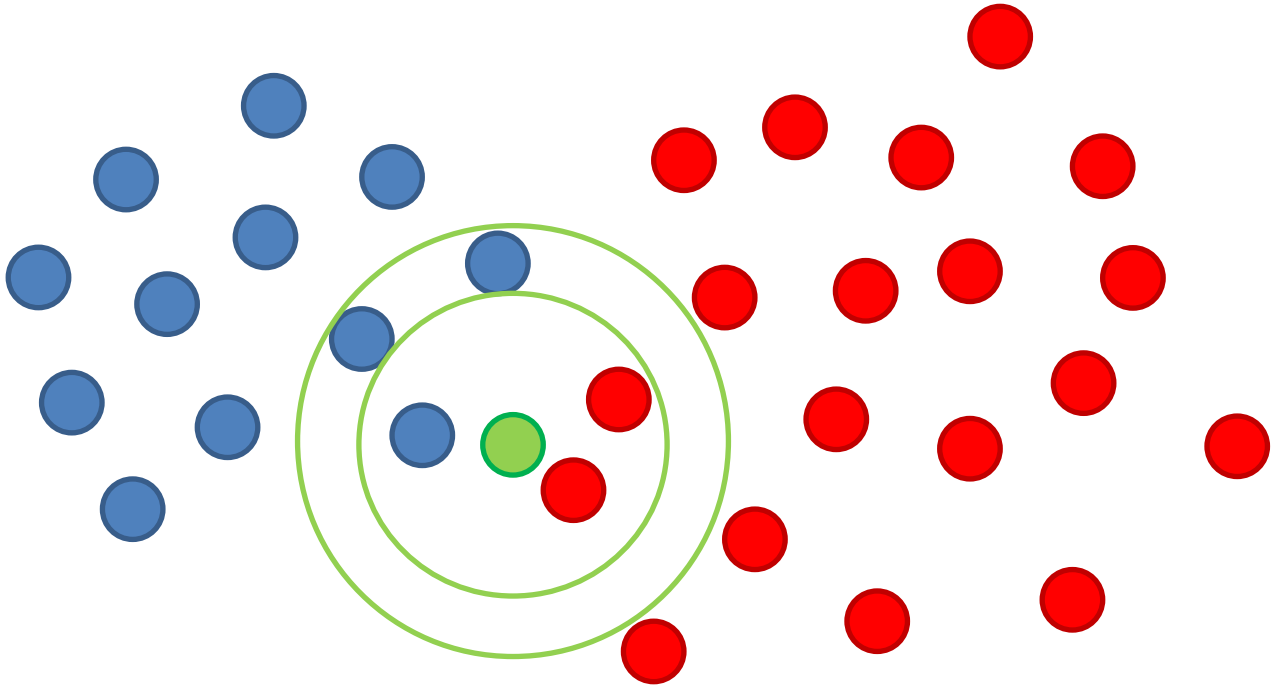
K = 4

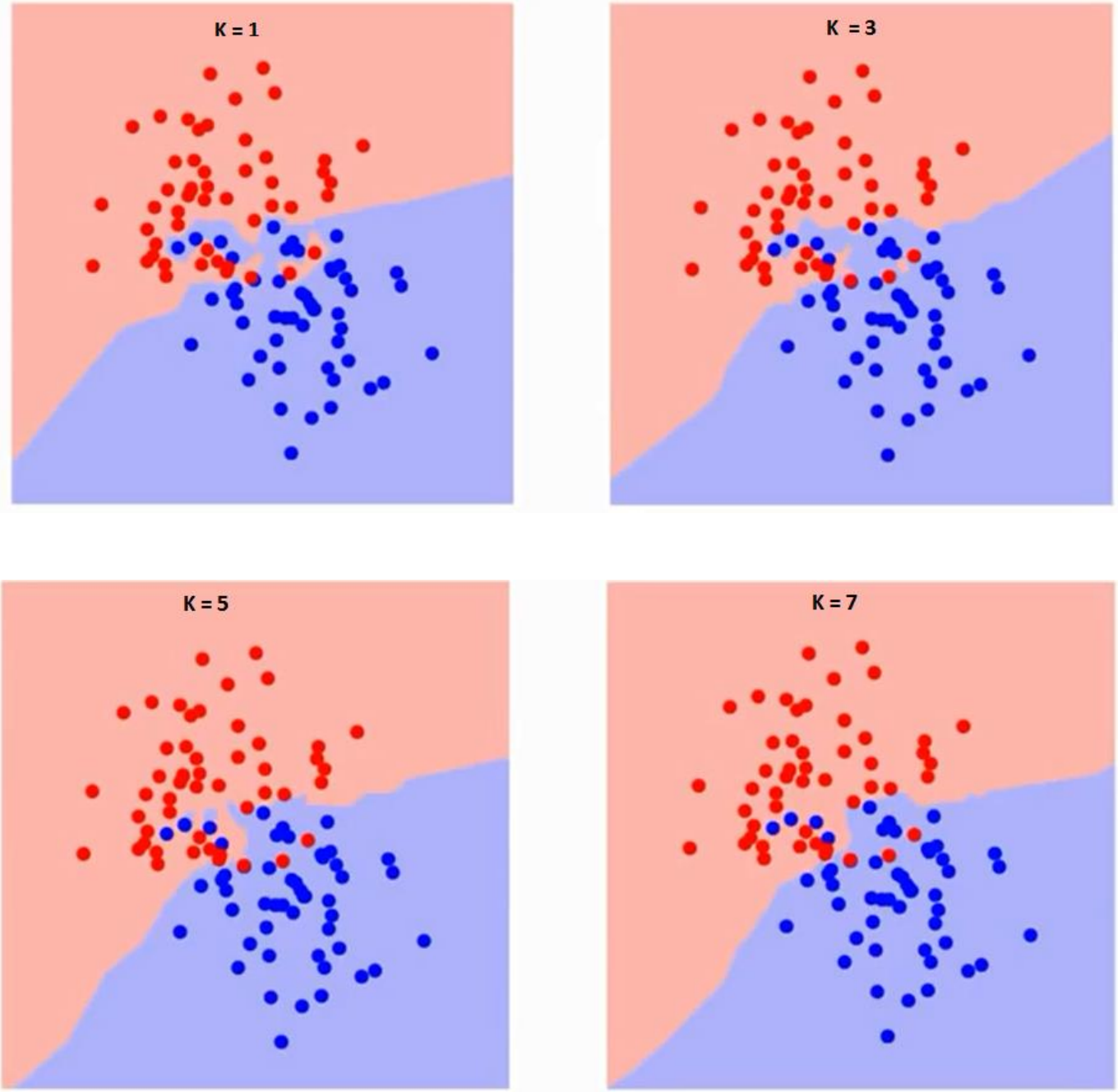


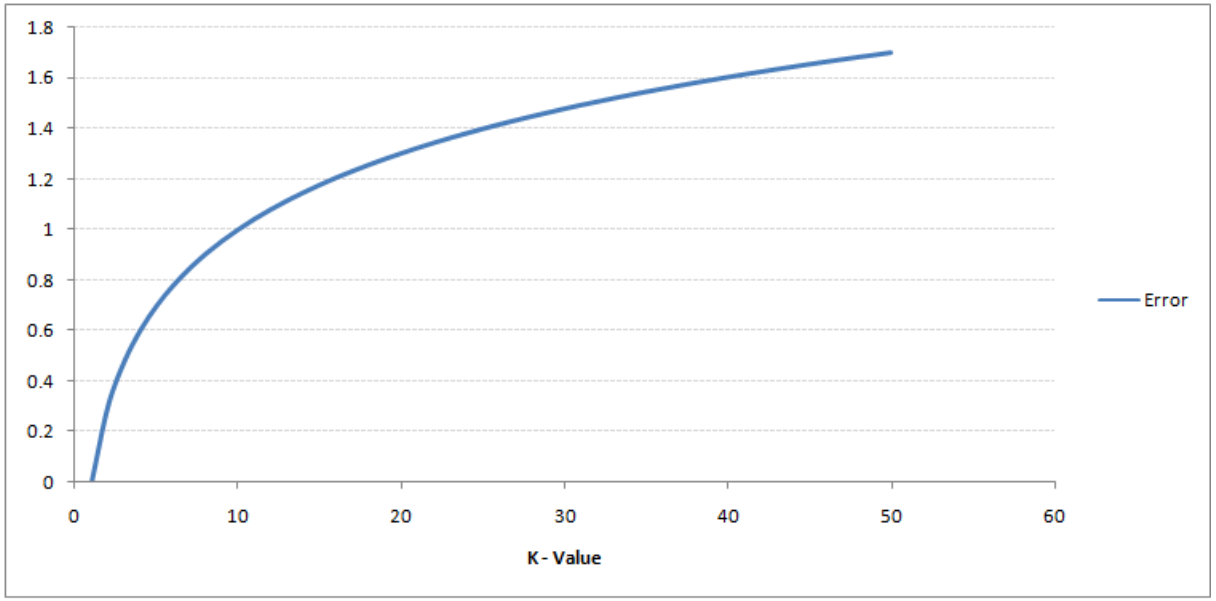
kNN

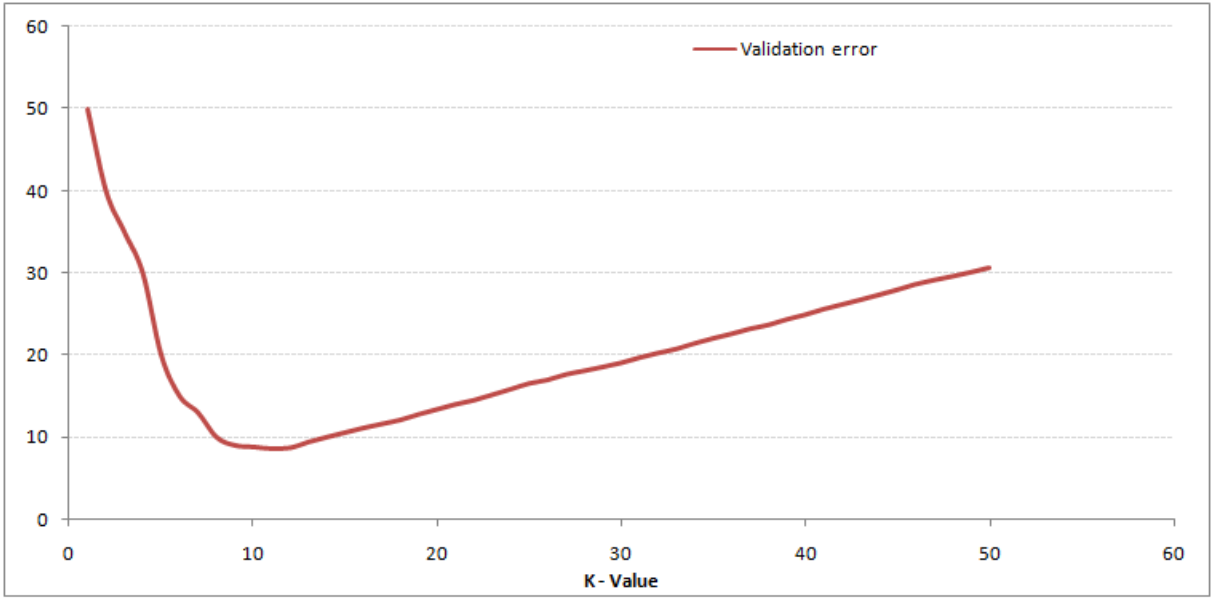


kNN

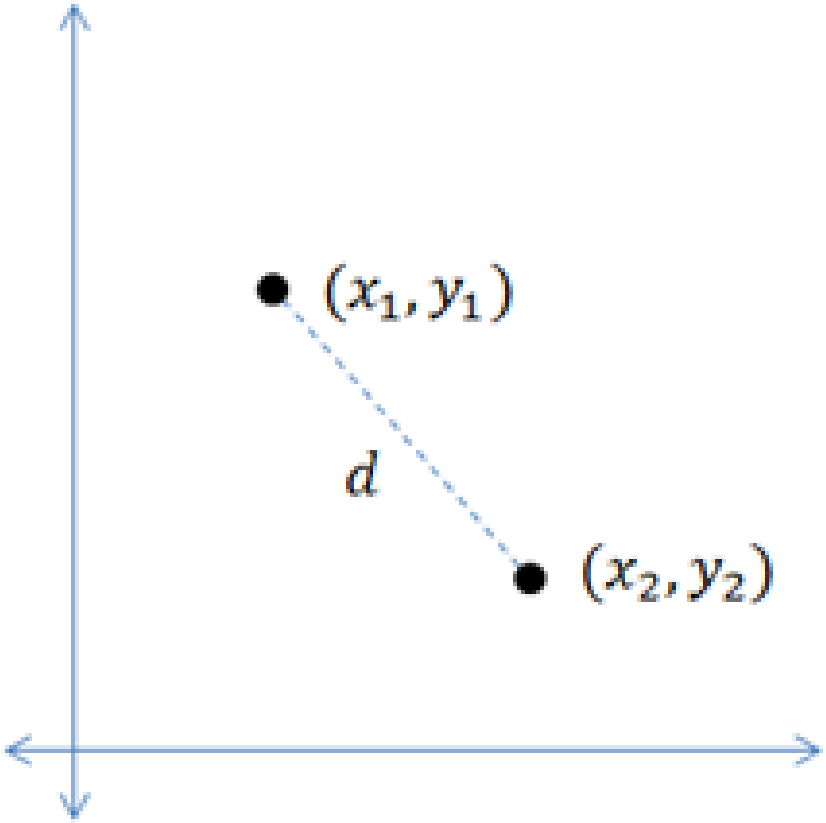



K = 1
K = 3
K = 5
K = 7


1.8
1.6
1.4
1.2
1
0.8
0.6
0.4
0.2
0
0
10
20
30
40
50
60
K - Value
Error


60
50
40
30
20
10
0
0
10
20
30
40
50
60
K - Value
Validation error

Euclidean Distance



$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

*k*NN: Pseudocode

k-Nearest Neighbor
Classify ($\mathbf{X}, \mathbf{Y}, x$) // $\mathbf{X}$: training data, $\mathbf{Y}$: class labels of $\mathbf{X}$, x: unknown sample
for $i = 1$ **to** m **do**
  Compute distance $d(\mathbf{X}_i, x)$
end for
Compute set I containing indices for the k smallest distances $d(\mathbf{X}_i, x)$.
return majority label for $\{\mathbf{Y}_i$ where $i \in I\}$

*k*NN: Pseudocode

```
/*
*    X: training data,                Y: class labels of X
*    x_t: test sample,                y_t: class label of x_t
*/
function kNNClassify(X, Y, x_t, k) {
    distancePairList[]
    for x_i, y_i in X, Y {
        d_i = computeDistance(x_i, x_t)
        distancePairList.add(x_i, y_i, d_i)
    }
    sortByDistance(distancePairList)
    kNeighborsList = distancePairList.getFirst(k)
    y_t = getMajorityLabel(kNeighborsList)
    return y_t
}
```

Data Split

	index	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	
0	73	6.1	2.8	4.7	1.2	1.0	TRAIN
1	18	5.7	3.8	1.7	0.3	0.0	
2	118	7.7	2.6	6.9	2.3	2.0	
3	78	6.0	2.9	4.5	1.5	1.0	
4	76	6.8	2.8	4.8	1.4	1.0	
5	31	5.4	3.4	1.5	0.4	0.0	
6	64	5.6	2.9	3.6	1.3	1.0	
7	141	6.9	3.1	5.1	2.3	2.0	
8	68	6.2	2.2	4.5	1.5	1.0	TEST
9	82	5.8	2.7	3.9	1.2	1.0	
		X				y	

Data Split

	index	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	
0	73	6.1	2.8	4.7	1.2	1.0	TRAIN
1	18	5.7	3.8	1.7	0.3	0.0	
2	118	7.7	2.6	6.9	2.3	2.0	
3	78	6.0	2.9	4.5	1.5	1.0	
4	76	6.8	2.8	4.8	1.4	1.0	
5	31	5.4	3.4	1.5	0.4	0.0	
6	64	5.6	2.9	3.6	1.3	1.0	VAL
7	141	6.9	3.1	5.1	2.3	2.0	
8	68	6.2	2.2	4.5	1.5	1.0	TEST
9	82	5.8	2.7	3.9	1.2	1.0	
			X				y

Homework: *k*NN

Implement the k-Nearest Neighbors algorithm using the following data:

- https://archive.ics.uci.edu/ml/datasets/iris

Addition to the task:

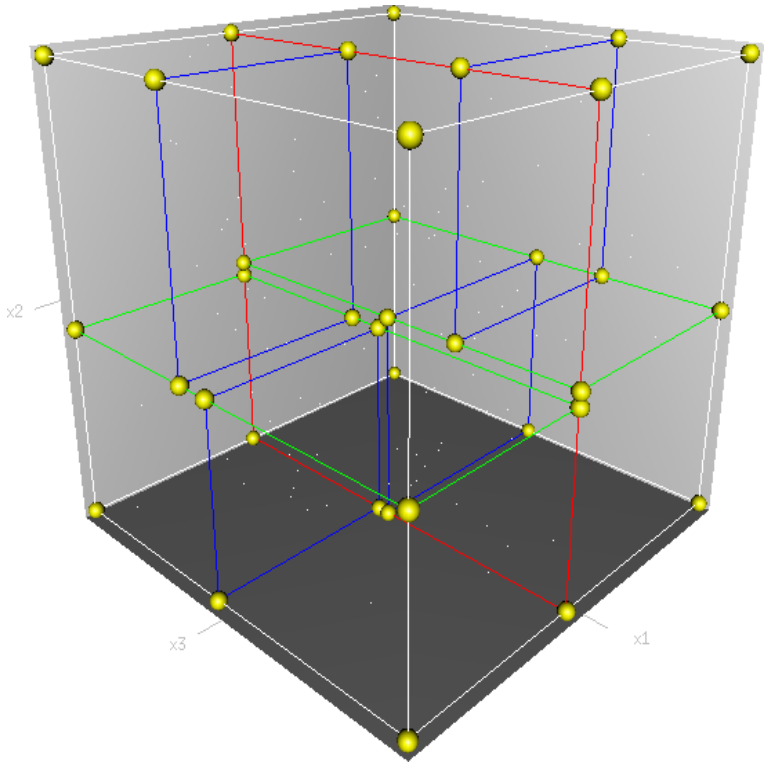
- To test the algorithm, apply 10-fold cross-validation (divide the data randomly into 10 sets and do 10 trainings and tests of the model, each time using one of the sets for testing and the other 9 for training).
- Display the Accuracy metric for each of the 10 trainings, as well as their arithmetic mean (for a summary assessment of the classifier).

** Make the algorithm accepts the parameter k as input.*

*** Store the dataset into k-D Tree.*

k-D Tree

- ***k*-d tree** (short for *k*-dimensional tree)
- *k*-D Trees and *K*NN Searches



Type	Multidimensional BST
Invented	1975
Invented by	Jon Louis Bentley

Time complexity in big O notation

Algorithm	Average	Worst case
Space	$O(n)$	$O(n)$
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$